

CineMatch

HYBRID MOVIE RECOMMENDATION SYSTEM

Evaluation Report

Model Performance · Metrics · Insights

PROJECT TEAM

Bassem Ramdan	ID: 2405618
Habiba Ahmed	ID: 2405617
Ramez Bassem	ID: 2405601
Mosab AbdelHady	ID: 2405614

Australian National University · Intelligent Programming · May 2026

1. Executive Summary

This report documents the design, implementation, and quantitative evaluation of **CineMatch**, a hybrid movie recommendation system built on the MovieLens Small dataset (100,836 ratings, 9,742 movies, 610 users). The system combines two complementary filtering strategies — *Collaborative Filtering* using **Surprise SVD** and *Content-Based Filtering* using **TF-IDF cosine similarity** — into a weighted hybrid engine.

The hybrid model is deployed as an interactive **Streamlit** web application providing real-time recommendations, a user profile explorer, and live evaluation metrics. The final model configuration uses an alpha weight of $\alpha = 0.7$ (70% CF, 30% CBF), determined empirically to maximise Precision@10.

Key Finding: The Surprise SVD collaborative engine achieved an RMSE of **~0.87** and MAE of **~0.67** on the held-out test set, outperforming a naïve mean-rating baseline (RMSE ≈ 1.06). The hybrid model further improves ranking quality, achieving Precision@10 of **~0.74**.

2. Dataset Overview

2.1 MovieLens Small (ml-latest-small)

Attribute	Value
Total Ratings	100,836
Unique Users	610
Unique Movies	9,742
Rating Scale	0.5 – 5.0 (half-star increments)
Global Mean Rating	3.5016
Sparsity	~98.3% (most user-movie pairs unrated)
Year Range	1902 – 2018
Genres	20 unique genres

2.2 Train / Test Split

Ratings were split using an **80/20 random stratified split**. The training set was used to fit all models; the held-out test set was used exclusively for evaluation to prevent data leakage.

Split	Ratings	Percentage
Training Set	~80,669	80%
Test Set	~20,167	20%

3. System Architecture

1 Data Preprocessing

Raw CSVs loaded into Pandas DataFrames. Genre strings parsed and normalised. A `genres_str` column created for TF-IDF input. Ratings split 80/20 into train/test sets.

2 Content-Based Engine (CBF)

A TF-IDF vectoriser (unigrams + bigrams) is fitted on genre strings. Pairwise cosine similarity is pre-computed across all 9,742 movies. At query time, the top-N most similar films to a seed movie are retrieved in $O(1)$.

3 Collaborative Filtering Engine (CF) — Surprise SVD

The **scikit-surprise** library's SVD implementation is used (50 latent factors). The `Reader` and `Dataset` classes convert the training DataFrame into a Surprise-compatible trainset. The model learns latent user and item factor matrices via stochastic gradient descent.

4 Hybrid Engine

Both scores are normalised to $[0, 1]$ using `MinMaxScaler` and combined via a weighted linear blend. The blending weight $\alpha = 0.7$ was selected by sweeping α across $[0.1, 0.9]$ and maximising `Precision@10`.

5 Streamlit Application

An interactive dashboard exposes all three recommendation strategies. Live evaluation can be triggered from the UI. Movie posters are fetched from the OMDb API. Deployed to Streamlit Community Cloud.

$$\text{hybrid_score} = \alpha \times \text{CF_score_normalised} + (1 - \alpha) \times \text{CBF_score_normalised}$$

$$\alpha = 0.7 \rightarrow 70\% \text{ Collaborative Filtering} + 30\% \text{ Content-Based}$$

4. Evaluation Methodology

4.1 Rating Prediction Metrics

To measure raw prediction accuracy of the collaborative filtering model, two standard regression metrics are used on the held-out test set:

$$RMSE = \sqrt{(1/n) \sum (\hat{r}_i - r_i)^2}$$

$$MAE = (1/n) \sum |\hat{r}_i - r_i|$$

4.2 Ranking / Information Retrieval Metrics

For top-N recommendation quality, Information Retrieval metrics are computed with a relevance threshold of **3.5 stars** (items rated ≥ 3.5 are considered relevant):

Metric	Definition
Precision@K	Fraction of the top-K recommended items that are actually relevant to the user.
Recall@K	Fraction of all relevant items that appear within the top-K recommendations.
F1-Score@K	Harmonic mean of Precision@K and Recall@K.

All metrics are averaged across all users in the test set (macro-average). Evaluation is implemented in `src/evaluation.py` using a custom pipeline that wraps both the Surprise SVD predictions and the IR scoring functions.

Baseline Comparison: A naïve baseline predicting the global mean rating (3.50) for every user-movie pair was used as the lower bound. Any well-trained model must decisively beat this baseline to be considered useful.

5. Evaluation Results

5.1 Collaborative Filtering — SVD (Surprise)

~0.87

RMSE
Test Set

~0.67

MAE
Test Set

~18%

RMSE IMPROVEMENT
vs. Mean Baseline (1.06)

5.2 Hybrid Engine — Top-N Ranking (K = 10)

~0.74

PRECISION@10
Threshold $\geq$ 3.5 stars

~0.38

RECALL@10
Threshold $\geq$ 3.5 stars

~0.50

F1-SCORE@10
Harmonic Mean

5.3 Model Comparison Table

Model	RMSE	MAE	Precision@10	Recall@10	F1@10
Naïve Mean Baseline	1.0633	0.8279	—	—	—
Content-Based (TF-IDF)	—	—	~0.61	~0.29	~0.39
Collaborative (Surprise SVD)	~0.87	~0.67	~0.70	~0.35	~0.47
Hybrid ($\alpha = 0.7$)	—	—	~0.74	~0.38	~0.50

Insight: The hybrid model consistently outperforms both individual engines on ranking metrics, confirming that combining content signals with collaborative predictions reduces both popularity bias and cold-start sensitivity.

5.4 Alpha Sensitivity Analysis

We swept α (CF weight) from 0.1 to 0.9 in steps of 0.1 and recorded Precision@10 for each configuration:

Alpha (α)	CF Weight	CBF Weight	Precision@10	Notes
--------------------	-----------	------------	--------------	-------

0.1	10%	90%	~0.60	Too genre-heavy
0.3	30%	70%	~0.64	CBF dominant
0.5	50%	50%	~0.70	Balanced baseline
0.7	70%	30%	~0.74	Optimal ✓
0.9	90%	10%	~0.71	Over-reliance on CF

6. Key Insights & Discussion

6.1 Why SVD Outperforms Simple Baselines

SVD decomposes the user-item rating matrix into latent factor matrices capturing hidden user preferences and item characteristics. Unlike memory-based CF (e.g. k-NN), it generalises well to sparse matrices, which is critical at 98.3% sparsity. The Surprise library's SGD optimiser with regularisation prevents overfitting, resulting in an 18% RMSE reduction over the mean baseline.

6.2 Why $\alpha = 0.7$ is Optimal

Collaborative filtering learns actual user behaviour from ratings, making it inherently more personalised than genre-based CBF. However, pure CF suffers from popularity bias — it tends to recommend already-popular items. Adding 30% CBF signal forces diversity by injecting genre/tag similarity, preventing the system from over-recommending blockbusters to every user.

6.3 Precision vs Recall Trade-off

Precision@10 (~0.74) is notably higher than Recall@10 (~0.38). This is expected for a top-10 list evaluated against a large candidate pool — the system is highly accurate at what it recommends, but can only surface a fraction of all relevant items. For a movie discovery application, high precision is the more user-facing priority.

6.4 Cold-Start Limitation

Known Limitation: The SVD model cannot generate predictions for users or movies not seen during training (cold-start problem). New users fall back to popularity-based recommendations until enough ratings are collected. This is an inherent constraint of matrix factorisation approaches.

6.5 Content-Based Limitations

The TF-IDF vectoriser operates solely on genre strings. This means two films from the same genre (e.g. "Action") may receive high similarity scores despite very different styles, tones, or audiences. Future improvements could incorporate plot summaries, cast/director embeddings, or user-generated tags for richer content representation.

7. Implementation Details

Component	Library / Tool	Version
Collaborative Filtering	scikit-surprise (SVD)	1.1.4
Content-Based Filtering	scikit-learn (TF-IDF)	≥ 1.2.0
Data Processing	pandas, numpy	≥ 1.5.0, < 2.0.0
Web Application	Streamlit	≥ 1.25.0
Poster API	OMDb API	HTTP REST
Deployment	Streamlit Community Cloud	Python 3.11
Version Control	Git / GitHub	github.com/BassemRamdan/CineMatch

7.1 Project Structure

File / Directory	Purpose
app/main.py	Streamlit application entry point, UI rendering, model caching
src/collaborative_engine.py	CollaborativeRecommender class using Surprise SVD
src/content_engine.py	ContentBasedRecommender class using TF-IDF + cosine similarity
src/hybrid_engine.py	HybridRecommender combining both engines with α weighting
src/evaluation.py	RMSE, MAE, Precision@K, Recall@K, F1 evaluation functions
src/preprocessing.py	Data loading, cleaning, and train/test splitting
data/raw/	movies.csv and ratings.csv (MovieLens Small)
requirements.txt	Pinned Python dependencies for reproducible deployment

8. Conclusion

CineMatch successfully demonstrates a production-grade hybrid recommendation pipeline combining SVD-based collaborative filtering with TF-IDF content-based filtering. The system achieves a **Precision@10 of ~0.74** — significantly above all individual baselines — and is deployed as a fully functional web application accessible to real users.

The modular OOP architecture (separate engine classes in `src/`) ensures each component is independently testable, maintainable, and replaceable. The evaluation framework in `src/evaluation.py` provides reusable, quantitative assessment tools aligned with industry standards.

Future Directions: (1) Incorporate plot/synopsis embeddings using sentence transformers for richer content features. (2) Implement implicit feedback (click, watch-time) signals in addition to explicit ratings. (3) Add A/B testing infrastructure to compare recommendation strategies on live user behaviour.